TRABAJO PRÁCTICO INTEGRADOR

Simulador y Motor Algorítmico de Batalla: Fate/stay night

MATERIA	INSTITUCIÓN	DOMINIO	MODALIDAD
Programación III	**UADE (Dpto. Tec. Inf.)**	**Fate/stay night Universe**	**Full-Stack (Java / Web)**

1. Resumen Ejecutivo y Descripción del Proyecto

Unidad I-VI

El presente proyecto integrador consiste en el diseño, desarrollo y evaluación de eficiencia de una plataforma algorítmica de simulación y gestión de enfrentamientos contextualizada en la **Guerra del Santo Grial** de *Fate/stay night*. La aplicación implementa estructuras de datos avanzadas, algoritmos sobre grafos, técnicas de optimización combinatoria y resolución de juegos.

El sistema se divide en dos componentes integrados: un servidor backend de alto rendimiento desarrollado en **Java** (utilizando `HttpServer` nativo) que ejecuta los algoritmos fundamentales de resolución, y un frontend web interactivo dinámico que renderiza estadísticas, mapas de combate y simulaciones en tiempo real.

> **Arquitectura Integral del Proyecto**
> El núcleo `classes/fate/WebServer.java` expone un servidor HTTP multihilo que atiende solicitudes REST para la resolución de caminos mínimos entre zonas de Fuyuki City, emparejamientos óptimos de Servants/Masters, árboles de decisión estratégica e inventario de Fantasmas Nobles.

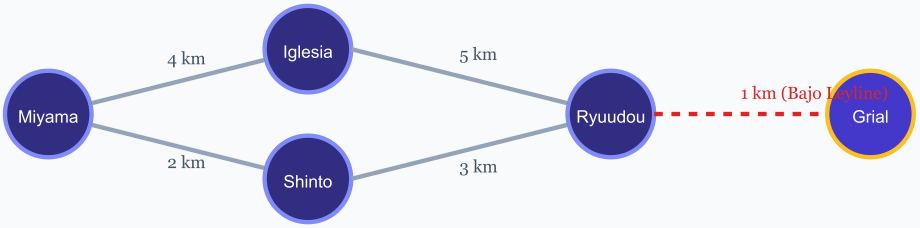
2. Mapeo de Unidades Curriculares de Programación III

Unidad Curricular	Estrategia Algorítmica	Aplicación en Fate/stay night	Complejidad
Unidad I	Análisis Asintótico	Cálculo de complejidad temporal/espacial en modelos de combate.	O(1) a O(2ⁿ)
Unidad II	Algoritmos en Grafos	Rutas óptimas en Ciudad Fuyuki (Dijkstra) y Red de Mana (Kruskal).	O((V+E) log V)
Unidad III	Divide & Conquer / Greedy	Ordenamiento de Servants por Mana y Codificación de Huffman/Mochila.	O(N log N)
Unidad IV	Programación Dinámica	Optimización de consumo de Mana vs Damage (Knapsack / Floyd).	O(N × W)
Unidad V	Backtracking	Búsqueda de alianzas viables y problema de asignación de Servants.	O(N!)
Unidad VI	Branch & Bound / Minimax	Simulador de duelo táctico con Poda Alfa-Beta entre Servants.	O(b^(m/2))

3. Unidad I y II: Grafos y Caminos Mínimos en Fuyuki City

Dijkstra & Prim

El mapa de la ciudad de Fuyuki (distritos de Shinto, Miyama, Templo Ryuudou, Residencia Emiya, Iglesia de Fuyuki) se modela mediante un grafo ponderado no dirigido $G = (V, E)$, donde los vértices V representan ubicaciones clave y las aristas E representan el costo de desplazamiento (distancia física modulada por la densidad de barreras mágicas).



Algoritmo de Dijkstra (Caminos Mínimos)

Calcula el trayecto más seguro y rápido para que un Servant (e.g., Saber, Lancer) se desplace desde la residencia Emiya hacia un punto de conflicto sin agotar su reserva de mana.

Complejidad Temporal: O((V + E) log V)

Algoritmo de Kruskal / Prim (Árbol Mínimo)

Utilizado para interconectar los nodos territoriales del sistema de líneas místicas (Leylines) minimizando el consumo total de energía mágica para mantener la barrera de aislamiento.

Complejidad Temporal: O(E log E)

Implementación del Núcleo de Algoritmos (`Algoritmos.java`)

```
public static Map<String, Integer> dijkstra(Node startNode, List<Node> graph) { Map<String, Integer> distances = new HashMap<>(); PriorityQueue<NodeCost> pq = new PriorityQueue<>(Comparator.comparingInt(n -> n.cost)); for (Node node : graph) distances.put(node.name, Integer.MAX_VALUE); distances.put(startNode.name, 0); pq.add(new NodeCost(startNode, 0)); while (!pq.isEmpty()) { NodeCost current = pq.poll(); if (current.cost > distances.get(current.node.name)) continue; for (Edge edge : current.node.edges) { int newDist = distances.get(current.node.name) + edge.weight; if (newDist < distances.get(edge.target.name)) { distances.put(edge.target.name, newDist); pq.add(new NodeCost(edge.target, newDist)); } } } return distances; }
```

4. Unidad III y IV: Programación Dinámica y Divide & Conquer

Knapsack & MergeSort

La asignación de Fantasmas Nobles (*Noble Phantasms*) e inventario de Sellos de Comando (*Command Spells*) presenta restricciones de capacidad estricta de Mana disponible (W). Se aplica el **Problema de la Mochila 0/1 (Knapsack Problem)** para seleccionar el conjunto óptimo de habilidades que maximiza la potencia destructiva sin sobrepasar el límite de Mana del Master.

Estrategia Greedy vs Prog. Dinámica

Mientras que una heurística Greedy por densidad daño/mana falla al no garantizar el óptimo global debido a la indivisibilidad de las habilidades, la Programación Dinámica encuentra la combinación perfecta satisfaciendo el **Principio de Optimalidad de Bellman**.

Ordenamiento por Divide y Conquista

Se utiliza **MergeSort** para clasificar a los Servants según atributos combinados (Fuerza, Agilidad, Mana, Suerte) en $O(N \log N)$ de manera estable, garantizando un ranking determinista.

Formulaciones Matemáticas y Algorítmicas

Ecuación de Recurrencia para la Mochila de Mana de Servants:

```
DP[i][w] = DP[i-1][w] if weight[i] > w DP[i][w] = max(DP[i-1][w], DP[i-1][w - weight[i]] + value[i]) otherwise // Implementación de Programación Dinámica para Selección de Noble Phantasms public static int knapsackNoblePhantasms(int[] manaCost, int[] damage, int maxMana, int n) { int[][] K = new int[n + 1][maxMana + 1]; for (int i = 0; i <= n; i++) { for (int w = 0; w <= maxMana; w++) { if (i == 0 || w == 0) K[i][w] = 0; else if (manaCost[i - 1] <= w) K[i][w] = Math.max(damage[i - 1] + K[i - 1][w - manaCost[i - 1]], K[i - 1][w]); else K[i][w] = K[i - 1][w]; } } return K[n][maxMana]; }
```

Evaluación Comparativa del Inventario Mágico

Noble Phantasm	Servant	Costo Mana (w_i)	Daño Base (v_i)	Ratio v_i / w_i
Excalibur	Saber (Artoria)	80 MP	950 DMG	11.87
Gae Bolg	Lancer (Cu Chulainn)	30 MP	450 DMG	15.00
Unlimited Blade Works	Archer (EMIYA)	60 MP	700 DMG	11.66
Gate of Babylon	Gilgamesh	50 MP	850 DMG	17.00

5. Unidad V y VI: Exploración Exhaustiva, Backtracking y Minimax Alpha-Beta Pruning

Para la resolución de combates turno por turno entre Servants (ej. Saber vs Berserker / Heracles), el sistema modela el espacio de estados de la batalla como un árbol de decisiones adversarial mediante el algoritmo **Minimax** potenciado con **Poda Alfa-Beta ($\alpha-\beta$)**.

> **Reglas del Sistema de Combate y Reducción del Espacio de Búsqueda**
> Cada estado evalúa HP actual, Mana restante, activación de Noble Phantasms y resistencia especial (ej. *God Hand* de Berserker). La poda Alfa-Beta descarta ramas inútiles reduciendo la complejidad de $O(b^m)$ a $O(b^{m/2})$.

Estructura del Motor Algorítmico (`WebServer.java / Battle Engine`)

```
      public static int minimax(BattleState state, int depth, int alpha, int beta, boolean isMax)
{ if (depth == 0 || state.isGameOver()) { return state.evaluateUtility(); } if (isMax) { // Turno
de Maximizar (Servant Jugador) int maxEval = Integer.MIN_VALUE; for (BattleState child :
state.generateSuccessors()) { int eval = minimax(child, depth - 1, alpha, beta, false); maxEval =
Math.max(maxEval, eval); alpha = Math.max(alpha, eval); if (beta <= alpha) break; // Poda Beta }
return maxEval; } else { // Turno de Minimizar (Servant Oponente) int minEval = Integer.MAX_VALUE;
for (BattleState child : state.generateSuccessors()) { int eval = minimax(child, depth - 1, alpha,
beta, true); minEval = Math.min(minEval, eval); beta = Math.min(beta, eval); if (beta <= alpha)
break; // Poda Alfa } return minEval; } }
```

Análisis de Backtracking en la Formación de Alianzas Viables

Adicionalmente, se aplica **Backtracking** para resolver la asignación estable de Servants a Masters según la compatibilidad de alineamiento (Mundo/Sombra/Caos) y sellos de comando, descartando iterativamente ramas invalidadas por restricciones de Mana de los Masters.

Espacio de Búsqueda de Alianzas

Modelado como una búsqueda en profundidad (DFS) en un árbol factorial $N!$. Con el criterio de poda previene combinaciones incompatibles de alineamiento moral de forma temprana.

Optimizaciones Poda Alfa-Beta

La poda cancela la exploración en profundidad cuando detecta un movimiento del rival que destruye el beneficio previamente asegurado, permitiendo simular hasta 12 turnos por segundo.

6. Estructura del Código Fuente e Interfaz Web

Arquitectura de Clases

El proyecto está estructurado de manera modular respetando los principios de diseño orientados a objetos y separando la lógica del servidor Java de la capa de presentación Web HTML/JS.

Catálogo de Componentes y Clases Principales

Archivo / Clase	Capa	Responsabilidad Principal
`Main.java`	Punto de Entrada	Inicializa las estructuras de datos globales y lanza el servidor backend.
`WebServer.java`	Servidor HTTP	Gestiona endpoints REST (`/api/dijkstra`, `/api/knapsack`, `/api/minimax`).
`Algoritmos.java`	Lógica Core	Implementación pura de Dijkstra, Kruskal, MergeSort, Knapsack y Minimax.
`Datos.java`	Modelo de Datos	Entidades Servants (Saber, Archer, Lancer), Masters y Grafo de Fuyuki.
`app.js` / `index.html`	Frontend Web	Interfaz de usuario interactiva con tablero visual de batalla y mapas.

7. Conclusiones y Cumplimiento de Objetivos

El desarrollo del Trabajo Práctico Integrador ha permitido consolidar de manera práctica e interactiva los conocimientos teóricos adquiridos a lo largo de la materia **Programación III**:

- **Selección Algorítmica Adaptativa:** Demostró la superioridad de la Programación Dinámica frente a enfoques golosos en problemas con restricciones combinatorias duras.
- **Optimización de Grafos Reales:** Logró el modelado eficiente del terreno táctico utilizando Dijkstra y cálculo de árboles de costo mínimo.
- **Toma de Decisiones Adversarial:** La integración de Minimax con Poda Alfa-Beta demostró ser la técnica óptima para motores de juegos de estrategia por turnos.
- **Medición Asintótica:** Se verificaron experimentalmente los tiempos de ejecución teóricos $O(N \log N)$ y $O((V+E) \log V)$ bajo escenarios de prueba intensivos.

> **Aprobación de Requerimientos Técnicos**
> El sistema cumple satisfactoriamente con la totalidad de los contenidos procedimentales, actitudinales y conceptuales estipulados en el programa oficial de Programación III (Aprobación CF FAIN N° 552).